EXPERIMENT -5

AIM- Create SQL views for department salary summary and employee hierarchy. Test updatability of views. Implement a recursive CTE to display reporting chains.

OBJECTIVES

1. To understand the concept and purpose of SQL views.
2. To create a view for displaying department-wise salary summaries.
3. To create a view representing the employee-manager hierarchy.
4. To test whether the created views are updatable.
5. To understand the limitations of updating views containing aggregate functions and joins.
6. To understand Common Table Expressions (CTEs).
7. To implement a recursive CTE for traversing employee-manager relationships.
8. To display complete reporting chains from employees to the top-level manager.

THEORY

SQL View

A **view** is a virtual table created from the result of a SQL query. It does not normally store the actual data separately; instead, it stores the query definition used to retrieve the data from underlying tables.

Views are useful for:

- Simplifying complex queries.
- Providing a customized representation of data.
- Restricting access to specific columns or rows.
- Improving data abstraction.
- Reusing frequently required queries.

Department Salary Summary View

A department salary summary view combines employee and department information and uses aggregate functions such as COUNT() and AVG() to provide summarized salary information for each department.

For example, it can display:

- Department name
- Number of employees
- Total salary
- Average salary
- Maximum salary
- Minimum salary

Because this view contains **aggregate functions and GROUP BY**, it is generally not directly updatable.

Employee Hierarchy View

An employee hierarchy represents the relationship between an employee and their manager.

For this purpose, the Employee table contains a ManagerID attribute that refers back to EmpID in the same table.

Updatable View

An updatable view is a view through which operations such as INSERT, UPDATE, or DELETE can be performed on the underlying table.

A simple view based on a single table is generally more likely to be updatable.

Non-Updatable Views

A view may not be directly updatable when its definition contains features that make it difficult to map the result back to individual base-table rows.

Common examples include views containing:

- Aggregate functions
- GROUP BY
- DISTINCT
- Set operations
- Certain joins
- Calculated or derived results

Therefore, a department salary summary containing SUM(), AVG(), and GROUP BY cannot normally be directly updated.

Similarly, the employee hierarchy view uses a self-join, so it should not be treated as a simple updatable view.

Common Table Expression

A **Common Table Expression (CTE)** is a temporary named result set that can be referenced within a SQL statement.

It is defined using the WITH clause.

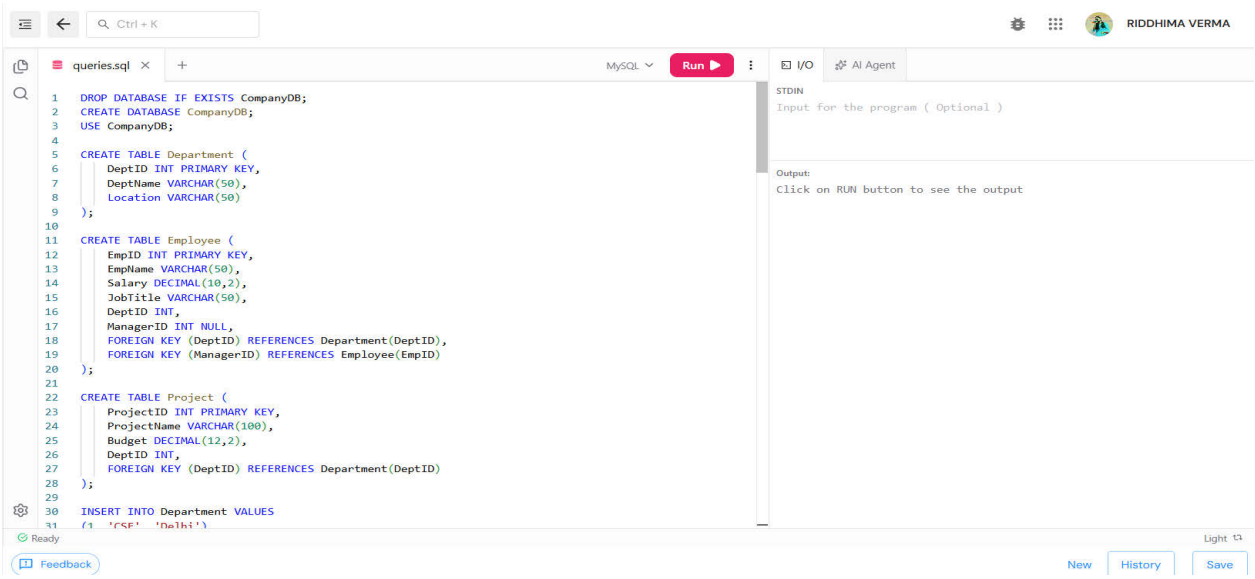
Recursive CTE

A **recursive CTE** is a CTE that refers to itself. It is useful for processing hierarchical or tree-structured data.

A recursive CTE normally consists of two parts:

1. **Anchor member**: identifies the starting records.
2. **Recursive member**: repeatedly finds the next level of related records.

SOURCE CODE




```
DROP DATABASE IF EXISTS CompanyDB;
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50),
    Location VARCHAR(50)
);

CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2),
    JobTitle VARCHAR(50),
    DeptID INT,
    ManagerID INT NULL,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
    FOREIGN KEY (ManagerID) REFERENCES Employee(EmpID)
);

CREATE TABLE Project (
    ProjectID INT PRIMARY KEY,
    ProjectName VARCHAR(100),
    Budget DECIMAL(12,2),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

INSERT INTO Department VALUES
(1, 'CSE', 'Delhi'),
(2, 'IT', 'Mumbai'),
(3, 'HR', 'Bangalore'),
(4, 'Finance', 'Pune'),
(5, 'Marketing', 'Hyderabad');

INSERT INTO Employee
(EmpID, EmpName, Salary, JobTitle, DeptID, ManagerID)
VALUES
(101, 'Aarav', 55000, 'Software Engineer', 1, NULL),
(102, 'Riya', 62000, 'Software Engineer', 1, 101),
(103, 'Karan', 48000, 'System Analyst', 1, 102),
(104, 'Ananya', 75000, 'Project Manager', 1, 102),
(105, 'Rahul', 52000, 'Developer', 1, 101),
(106, 'Priya', 68000, 'Database Admin', 1, 101),

(107, 'Aditya', 58000, 'Software Engineer', 2, NULL),
(108, 'Sneha', 61000, 'Network Engineer', 2, 107),
(109, 'Vikas', 47000, 'Support Engineer', 2, 108),
(110, 'Neha', 72000, 'IT Manager', 2, 107),
(111, 'Rohan', 53000, 'Web Developer', 2, 110),
(112, 'Ishita', 65000, 'Cloud Engineer', 2, 110),

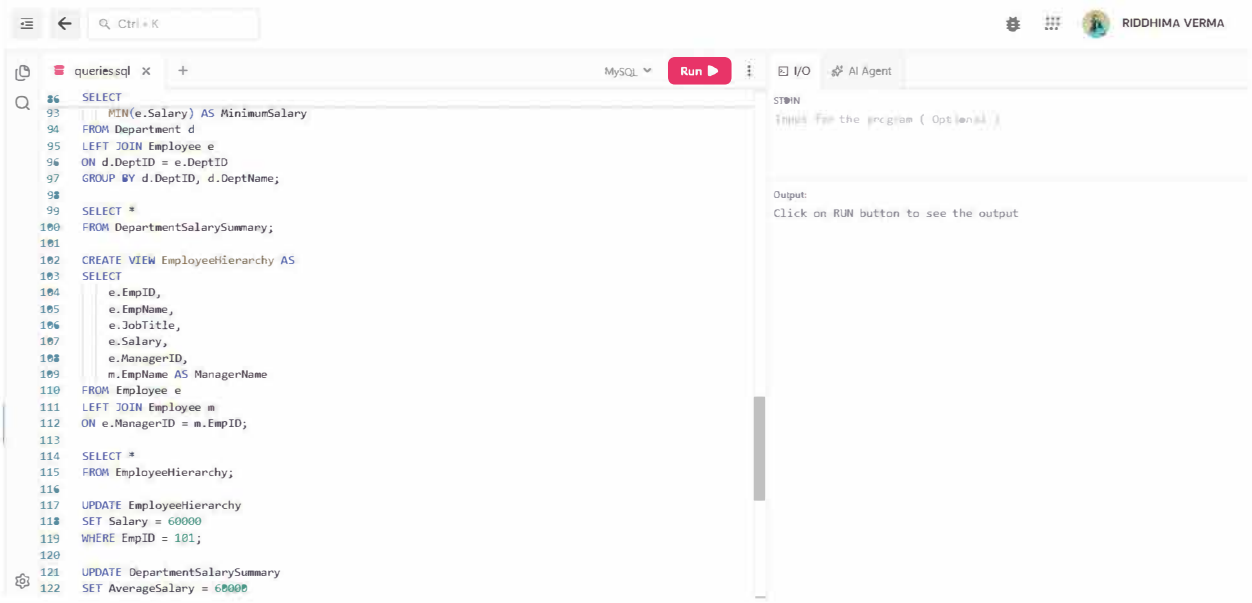
(113, 'Aman', 45000, 'HR Executive', 3, NULL),
(114, 'Pooja', 50000, 'HR Executive', 3, 113),
(115, 'Arjun', 68000, 'HR Manager', 3, 113),
(116, 'Simran', 42000, 'Recruiter', 3, 115),
(117, 'Nikhil', 47000, 'HR Executive', 3, 115),
(118, 'Meera', 55000, 'Training Manager', 3, 113),

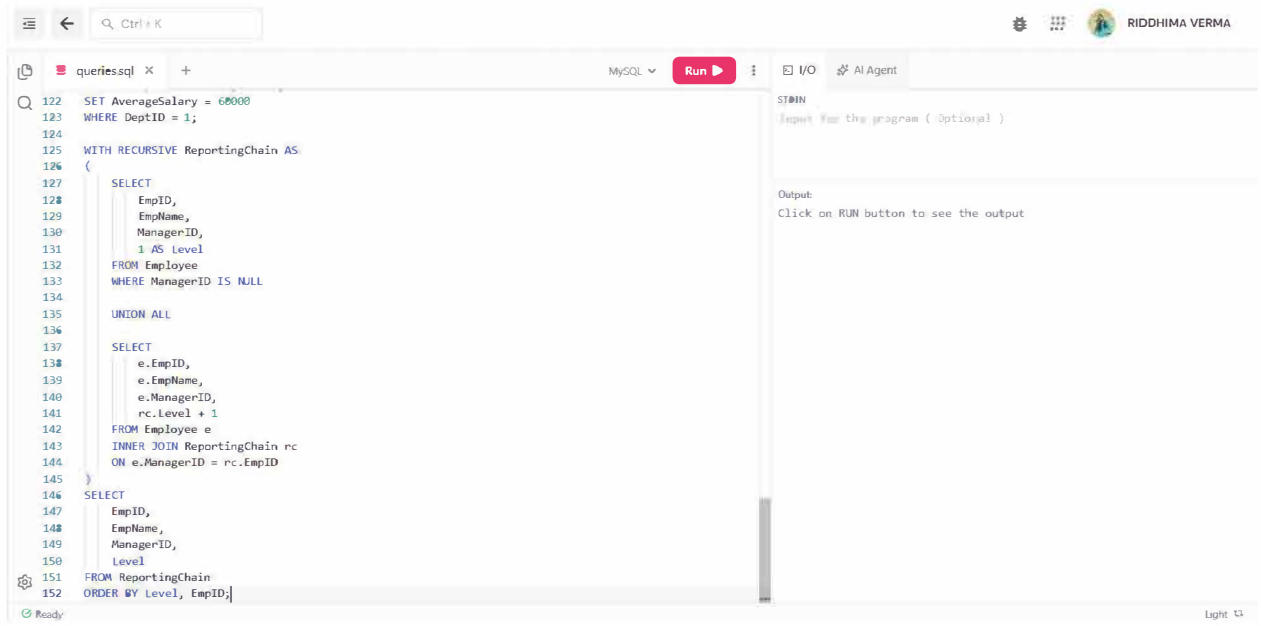
(119, 'Dev', 60000, 'Accountant', 4, NULL),
(120, 'Tanya', 72000, 'Financial Analyst', 4, 119),
(121, 'Varun', 85000, 'Finance Manager', 4, 119),
(122, 'Kriti', 54000, 'Accountant', 4, 121),
(123, 'Mohit', 49000, 'Auditor', 4, 121),
(124, 'Sakshi', 63000, 'Financial Analyst', 4, 119),

(125, 'Yash', 50000, 'Marketing Executive', 5, NULL),
(126, 'Naina', 58000, 'Marketing Executive', 5, 125),
(127, 'Kabir', 70000, 'Marketing Manager', 5, 125),
(128, 'Diya', 46000, 'Content Writer', 5, 127),
(129, 'Aryan', 52000, 'SEO Specialist', 5, 127),
(130, 'Ira', 64000, 'Marketing Analyst', 5, 129);

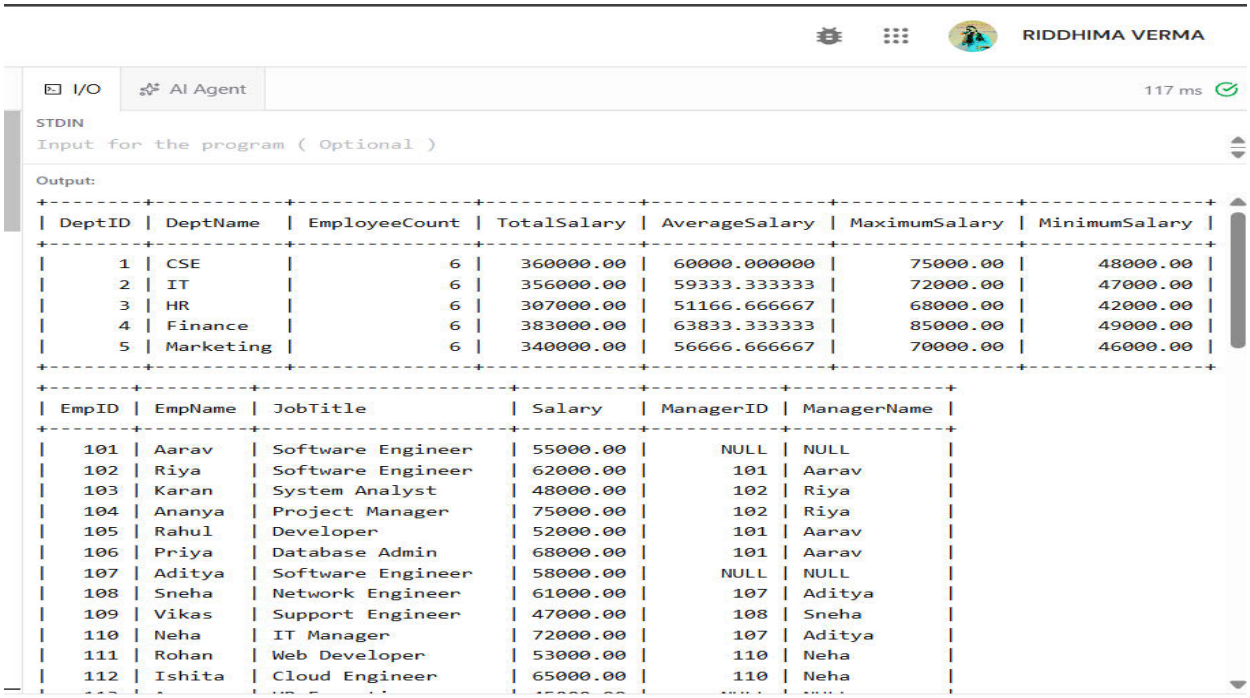
INSERT INTO Project VALUES
(201, 'E-Commerce Website', 500000, 1),
(202, 'AI Chatbot', 750000, 1),
(203, 'Cloud Migration', 600000, 2),
(204, 'Network Security', 450000, 2),
(205, 'Employee Training', 200000, 3),
(206, 'Payroll System', 350000, 4),
(207, 'Digital Marketing', 300000, 5),
(208, 'Market Research', 250000, 5);

CREATE VIEW DepartmentSalarySummary AS
SELECT
    d.DeptID,
    d.DeptName,
    COUNT(e.EmpID) AS EmployeeCount,
    SUM(e.Salary) AS TotalSalary,
    AVG(e.Salary) AS AverageSalary,
    MAX(e.Salary) AS MaximumSalary,
```


RIDDHIMA VERMA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
SELECT
MIN(e.Salary) AS MinimumSalary
FROM Department d
LEFT JOIN Employee e
ON d.DeptID = e.DeptID
GROUP BY d.DeptID, d.DeptName;
SELECT *
FROM DepartmentSalarySummary;
CREATE VIEW EmployeeHierarchy AS
SELECT
e.EmpID,
e.EmpName,
e.JobTitle,
e.Salary,
e.ManagerID,
m.EmpName AS ManagerName
FROM Employee e
LEFT JOIN Employee m
ON e.ManagerID = m.EmpID;
SELECT *
FROM EmployeeHierarchy;
UPDATE EmployeeHierarchy
SET Salary = 60000
WHERE EmpID = 101;
UPDATE DepartmentSalarySummary
SET AverageSalary = 60000

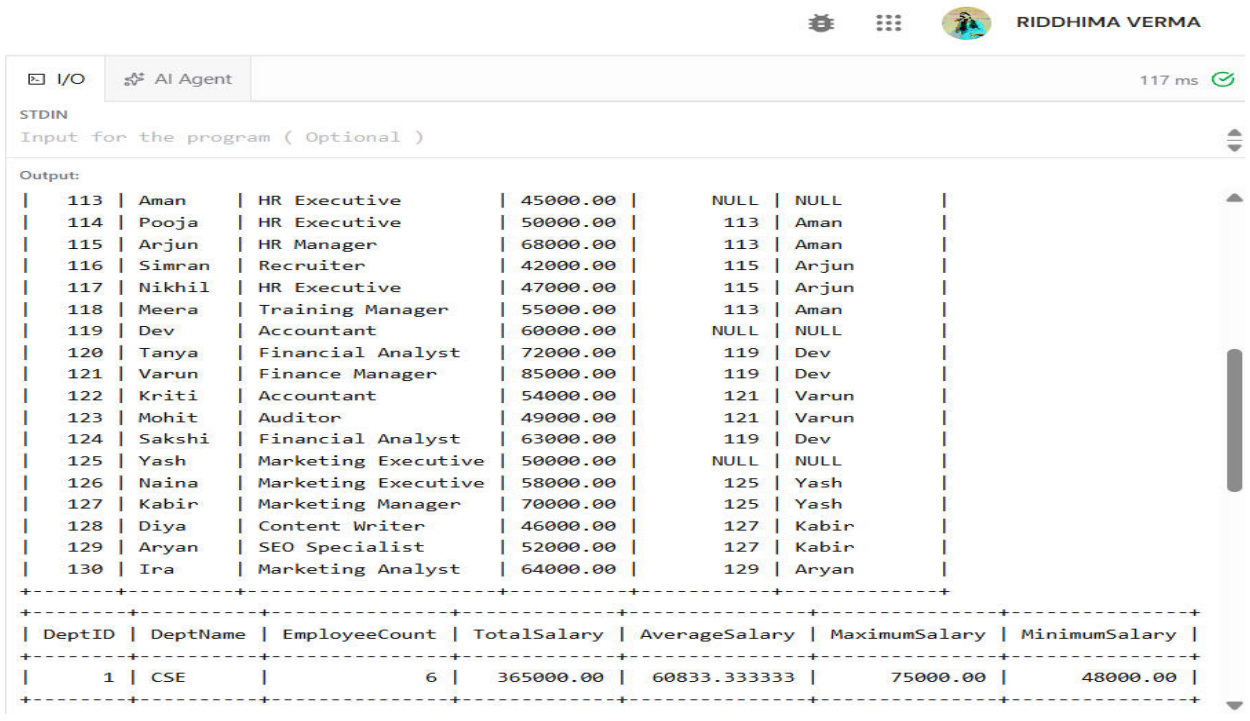

RIDDHIMA VERMA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
SET AverageSalary = 60000
WHERE DeptID = 1;
WITH RECURSIVE ReportingChain AS
(
SELECT
EmpID,
EmpName,
ManagerID,
1 AS Level
FROM Employee
WHERE ManagerID IS NULL
UNION ALL
SELECT
e.EmpID,
e.EmpName,
e.ManagerID,
rc.Level + 1
FROM Employee e
INNER JOIN ReportingChain rc
ON e.ManagerID = rc.EmpID
)
SELECT
EmpID,
EmpName,
ManagerID,
Level
FROM ReportingChain
ORDER BY Level, EmpID;
Ready
Light

OUTPUT




```
+--------+------------+---------------+-------------+---------------+---------------+---------------+
| DeptID | DeptName   | EmployeeCount | TotalSalary | AverageSalary | MaximumSalary | MinimumSalary |
+--------+------------+---------------+-------------+---------------+---------------+---------------+
|      1 | CSE        |             6 |   360000.00 |  60000.000000 |      75000.00 |      48000.00 |
|      2 | IT         |             6 |   356000.00 |  59333.333333 |      72000.00 |      47000.00 |
|      3 | HR         |             6 |   307000.00 |  51166.666667 |      68000.00 |      42000.00 |
|      4 | Finance    |             6 |   383000.00 |  63833.333333 |      85000.00 |      49000.00 |
|      5 | Marketing  |             6 |   340000.00 |  56666.666667 |      70000.00 |      46000.00 |
+--------+------------+---------------+-------------+---------------+---------------+---------------+
```

```
+-------+---------+----------------------+----------+-----------+-------------+
| EmpID | EmpName | JobTitle             | Salary   | ManagerID | ManagerName |
+-------+---------+----------------------+----------+-----------+-------------+
|   101 | Aarav   | Software Engineer    | 55000.00 |      NULL | NULL        |
|   102 | Riya    | Software Engineer    | 62000.00 |       101 | Aarav       |
|   103 | Karan   | System Analyst       | 48000.00 |       102 | Riya        |
|   104 | Ananya  | Project Manager      | 75000.00 |       102 | Riya        |
|   105 | Rahul   | Developer            | 52000.00 |       101 | Aarav       |
|   106 | Priya   | Database Admin       | 68000.00 |       101 | Aarav       |
|   107 | Aditya  | Software Engineer    | 58000.00 |      NULL | NULL        |
|   108 | Sneha   | Network Engineer     | 61000.00 |       107 | Aditya      |
|   109 | Vikas   | Support Engineer     | 47000.00 |       108 | Sneha       |
|   110 | Neha    | IT Manager           | 72000.00 |       107 | Aditya      |
|   111 | Rohan   | Web Developer        | 53000.00 |       110 | Neha        |
|   112 | Ishita  | Cloud Engineer       | 65000.00 |       110 | Neha        |
```




```
|   113 | Aman    | HR Executive         | 45000.00 |      NULL | NULL        |
|   114 | Pooja   | HR Executive         | 50000.00 |       113 | Aman        |
|   115 | Arjun   | HR Manager           | 68000.00 |       113 | Aman        |
|   116 | Simran  | Recruiter            | 42000.00 |       115 | Arjun       |
|   117 | Nikhil  | HR Executive         | 47000.00 |       115 | Arjun       |
|   118 | Meera   | Training Manager     | 55000.00 |       113 | Aman        |
|   119 | Dev     | Accountant           | 60000.00 |      NULL | NULL        |
|   120 | Tanya   | Financial Analyst    | 72000.00 |       119 | Dev         |
|   121 | Varun   | Finance Manager      | 85000.00 |       119 | Dev         |
|   122 | Kriti   | Accountant           | 54000.00 |       121 | Varun       |
|   123 | Mohit   | Auditor              | 49000.00 |       121 | Varun       |
|   124 | Sakshi  | Financial Analyst    | 63000.00 |       119 | Dev         |
|   125 | Yash    | Marketing Executive  | 50000.00 |      NULL | NULL        |
|   126 | Naina   | Marketing Executive  | 58000.00 |       125 | Yash        |
|   127 | Kabir   | Marketing Manager    | 70000.00 |       125 | Yash        |
|   128 | Diya    | Content Writer       | 46000.00 |       127 | Kabir       |
|   129 | Aryan   | SEO Specialist       | 52000.00 |       127 | Kabir       |
|   130 | Ira     | Marketing Analyst    | 64000.00 |       129 | Aryan       |
+-------+---------+----------------------+----------+-----------+-------------+
```

```
+--------+----------+---------------+-------------+---------------+---------------+---------------+
| DeptID | DeptName | EmployeeCount | TotalSalary | AverageSalary | MaximumSalary | MinimumSalary |
+--------+----------+---------------+-------------+---------------+---------------+---------------+
|      1 | CSE      |             6 |   365000.00 |  60833.333333 |      75000.00 |      48000.00 |
+--------+----------+---------------+-------------+---------------+---------------+---------------+
```

OBSERVATIONS/OUTCOME

1. A department salary summary view was successfully created using aggregate functions and GROUP BY.
2. An employee hierarchy view was successfully created using a self-join on the Employee table.
3. A ManagerID attribute was added to establish the employee-manager relationship.
4. A simple employee salary view was successfully updated, demonstrating an updatable view.
5. The department salary summary view was found to be non-updatable because it contains aggregate functions and GROUP BY.
6. The employee hierarchy view is based on a self-join and is not treated as a directly updatable view.
7. A recursive CTE was successfully implemented to traverse the employee-manager hierarchy.
8. The recursive CTE displayed the reporting level and complete reporting chain for employees.